

MOVIE RECOMMENDATION SYSTEM REPORT

I. INTRODUCTION

This recommendation engine plays a crucial role in filtering through vast amounts of data to deliver high-quality, personalized content to consumers. By continuously capturing and analyzing user behavior, it is able to fine-tune its suggestions, ensuring that the products or movies recommended align closely with individual preferences and are more likely to be appreciated and purchased.

The system leverages a blend of sophisticated algorithms and diverse strategies, including-

- **Content-Based Filtering-** By evaluating intrinsic features such as the director, actors, plot, and genre, this approach creates detailed profiles for each movie. It then matches these attributes with a user's historical preferences, ensuring that the recommended content closely fits their taste.
- **Collaborative Filtering (including Item-Based Filtering and SVD)-** This technique examines the interaction patterns between users and items (movies) to identify similarities. Through item-based filtering, the system recommends movies that are similar to those a user has enjoyed. Advanced techniques like Singular Value Decomposition (SVD) further enhance the accuracy by revealing hidden relationships in the user-item interaction matrix.
- **Random Walk-Based Methods-** Although not explicitly described in our key strategies above, random walk methods also offer a powerful way to capture deeper, more complex relationships by simulating movement through a network of users and movies. This can uncover less obvious connections that enhance the diversity of recommendations.

Together, these techniques allow the engine to progressively learn from user interactions and deliver recommendations that are not only popular but also finely tuned to individual tastes, thus improving both engagement and satisfaction.

II. DATASET

The project utilizes the MovieLens 100K dataset, which is one of the most widely used benchmarks for developing and testing movie recommendation systems. This dataset contains rich information that allows us to explore user preferences and movie attributes.

Dataset Composition

- Number of Users: 943
- Number of Movies: 1682
- Number of Ratings: 100,000 ratings (with ratings on a scale of 1 to 5)

Data Files and Their Features

1. Ratings Data (**u.data**):

This file contains the core user-movie interactions with the following fields:

- User ID: Unique identifier for each user.
- Movie ID: Unique identifier for each movie.
- Rating: A numeric value (from 1 to 5) assigned to the movie by the user.
- Timestamp: The time at which the rating was recorded.

2. Movie Metadata (**u.item**):

This file provides additional information about each movie including:

- Movie Title: The name of the movie.
- Release Date: When the movie was released.
- IMDB website: Link to more details about the movie.
- Genre Information: Binary flags indicating the movie's association with specific genres (e.g., Action, Comedy, Drama, etc.).

3. User Information (**u.user**):

This file contains demographic details of the users, such as:

- Age
- Gender
- Occupation
- Zip Code

Preprocessing

- **Data Loading-** Files (u.data, u.item, u.user) were loaded using appropriate delimiters and converted into pandas DataFrames.
- **Column Naming-** Meaningful column names were assigned for clarity (e.g., user_id, movie_id, rating, timestamp).
- **Data Cleaning-** Any inconsistencies or missing data points are addressed to ensure data quality. This includes validating data types and ensuring the integrity of the user and movie identifiers across different files. Although the dataset was clean, dropna() and drop_duplicates() were applied as standard practice.
- **Data Type Conversion-** Timestamps were converted from Unix format to datetime using pd.to_datetime() to support temporal analysis. This can facilitate temporal analysis and the tracking of trends over time.
- **Merging Datasets-** The three files were merged to create a unified DataFrame with user, movie, and rating information. This integration creates a comprehensive dataset for analysis, allowing the model to correlate user attributes with movie preferences.
- **Normalization/Scaling-** Depending on the analysis, ratings are normalized to adjust for individual user biases, or aggregated to examine overall patterns.

This comprehensive structure and careful preprocessing make the MovieLens 100K dataset an excellent resource for developing and evaluating recommendation algorithms.

III. METHODOLOGY

This project employs three recommendation techniques to generate personalized movie suggestions. Each technique leverages different paradigms for modeling user preferences.

3.1 User-Based Collaborative Filtering

This technique assumes that users with similar interaction histories will have similar preferences.

- Interaction Matrix Construction- Built a sparse binary interaction matrix
- The similarity between users is calculated based on their historical ratings using measures such as:
 - Cosine Similarity- Employed **cosine similarity** to compute user-to-user similarities
 - Pearson Correlation Coefficient: Measures the linear correlation between users' ratings while normalizing for differences in rating scales.

Once the similarity scores are calculated, a neighborhood of users with similar tastes is formed for each target user. Recommendations are then generated by aggregating the ratings from these similar users, giving higher weight to users who share greater similarity.

- **Recommendation Generation-**
 - For a target user 'u', identify top-K similar users based on cosine similarity.
 - Aggregate items interacted with by similar users, weighted by similarity, excluding those already interacted with by 'u'.
 - Rank items by weighted score and recommend top-N items.

3.2 Item-Based Collaborative Filtering

Item-based collaborative filtering centers on the similarity between items (movies) rather than users. To determine item similarity, the algorithm compares the rating patterns for pairs of movies using methods such as:

- **Item Similarity Calculation:**
 - Transposed the interaction matrix to obtain item vectors.
 - Cosine Similarity: Computes the cosine of the angle between two items' rating vectors, which highlights similarity regardless of the number of ratings.
 - Normalized Similarity: Mitigates user bias by subtracting the average rating for each user prior to computing similarity.

After calculating similarities, the system recommends movies that are most similar to those a user has already enjoyed. This approach benefits from the relative stability of item similarities over time compared to user preferences.

- **Recommendation Process-**

- For each item 'i' interacted with by a user, retrieve top similar items 'j'.
- Compute scores as a weighted sum of similarities and recommend items with highest scores not yet consumed.

3.3 Random-Walk-Based Pixie Algorithm

The Pixie algorithm employs a graph-based random walk strategy to generate recommendations. In this approach, the recommendation system is modeled as a bipartite graph where:

- Nodes represent users and movies.
- Edges represent interactions (e.g., ratings, clicks).

This approach treats the recommendation problem as a stochastic graph traversal.

1. **Graph Modeling-**

- Constructed a bipartite graph $G=(U \cup I, E)$ where:
 - U is the set of users,
 - I is the set of items,
 - E contains Edges (u, i) if user 'u' interacts with item 'i'
- Represented using an **adjacency list** for efficient traversal.

2. **Random Walk Procedure-**

- Initialized walks from items a user has interacted with.
- Each walk follows:
 - Select a neighbor of the current node uniformly at random.
 - Repeat for k steps or until a stopping criterion.
- Performed multiple such walks (e.g., 100–1000) for robustness.

3. **Scoring and Ranking-**

- Maintained a frequency count $f(i)$ for each item 'i' visited during walks.
- Normalized scores and ranked items based on visitation frequency.
- Filtered out already-interacted items and recommended the top-ranked unseen ones.

Why Graph-Based Approaches are Effective

- **Capturing Complex Relationships:** Graph-based methods uncover indirect associations between users and movies that traditional matrix-based approaches might overlook.

- Exploring Heterogeneity: Random walks navigate diverse paths through the graph, discovering items that may not be immediately obvious through direct similarity comparisons.
- Scalability and Adaptability: These methods inherently scale with large datasets and can continuously adapt to changes in user behavior and item popularity.

By initiating a random walk from a user node, the Pixie algorithm explores the neighborhood of interconnected movies and identifies candidates that are likely of interest. This probabilistic exploration balances the discovery of both popular and niche items, enhancing the overall diversity and relevance of the recommendations.

IV. IMPLEMENTATION

This section outlines the core architectural steps and algorithms used in code.

4.1 Building the Recommendation Functions

- **Data Preprocessing:**
 - Loaded user-item interaction data.
 - Created mappings: user $\rightarrow$ [items] and item $\rightarrow$ [users].
 - Encoded users and items using integer identifiers for sparse matrix efficiency.
- **Collaborative Filtering:**
 - Built CSR (Compressed Sparse Row) matrices for user and item vectors.
 - Used `sklearn.metrics.pairwise.cosine_similarity` for fast similarity computation.
 - Implemented logic to aggregate and score recommendation candidates.
- **Pixie Implementation:**
 - Graph was stored as a `defaultdict(list)` to represent an adjacency list.
 - Wrote a `random_walk()` function performing multiple walks starting from user history.

4.2 Adjacency List Graph Construction

- Data Integration- The first step involved merging the user-movie ratings with movie and user details. Once the merged data was available, it represented the relationships as a graph.
- Graph Structure-
 - Nodes- Represent both users and movies.
 - Edges- Represent interactions such as a rating or implicit feedback (e.g., a click). Each edge connects a user to a movie they have rated.
 - Adjacency List Representation- For each node, an entry was created in an adjacency list data structure. This allows for efficient look-ups of connected

nodes. For instance, for a given user, the list contains all movies rated by that user, and vice versa for movie nodes.

- Implementation Steps:
 - Iterate over the ratings data.
 - For each rating, add an edge from the user to the corresponding movie in the adjacency list.
 - If needed, create reciprocal edges (from movies to users) to allow bi-directional navigation in the graph.
 - Created a bidirectional graph such that:
 - $\text{graph}[\text{user_id}] = [\text{movie_id1}, \text{movie_id2}, \dots]$
 - $\text{graph}[\text{movie_id}] = [\text{user_id1}, \text{user_id2}, \dots]$

4.3 Random Walk Execution and Movie Ranking

- Random Walk Strategy-
 - Starting Point- The random walk is initiated from the **target user's node**. This ensures that the exploration begins within the user's immediate interaction neighborhood, maintaining relevance to their preferences.
 - Walk Parameters- Parameters such as the **number of steps (L)** and the **restart probability (α)** are defined to balance **exploration and exploitation**:
 - A higher number of steps allows deeper traversal into the graph.
 - Restart probability helps prevent the walk from straying too far from the user's interests, by occasionally jumping back to the starting node.
 - Walk Process:
 - **Step Selection**: At each iteration, the walker selects a random neighbor from the current node's adjacency list uniformly.
 - **Restart Mechanism**: With a probability ' α ', the walker returns to the starting node. Otherwise, it continues to a randomly chosen neighbor.
 - **Visit Count**: Each time a movie node is visited during the walk, a **visit counter** for that node is incremented. Over many walks, frequently visited nodes accumulate higher counts.
- Ranking Visited Movies-
 - Aggregation- Once all walks are completed, the visit counts for each movie node are aggregated to reflect how often they were encountered.
 - Scoring Function- A scoring mechanism is used to rank the visited movie nodes. Factors may include:
 - **Visit Frequency**: Higher frequency implies stronger association with the user's interests.

- **Diversity Regularization:** Penalizes over-popular items to ensure diversity, introducing lesser-known but relevant content.
- Final Ranking- Movies are sorted in descending order of their scores. The top-K items that the user hasn't interacted with yet are selected as final recommendations.

V. RESULTS AND EVALUATION

5.1 Example Output

To evaluate the effectiveness of different recommendation algorithms, we tested three collaborative filtering approaches—User-Based, Item-Based, and Random-Walk-Based (Pixie Algorithm)—on a movie recommendation task.

- **User-Based Collaborative Filtering** generated personalized suggestions by identifying users with similar preferences. For a user with a known interest in action and sci-fi genres, the system recommended *The Matrix*, *Inception*, and *Avengers*. However, when calling `recommend_movies_for_user(10, num=5)`, the top suggestions included lesser-known titles like *In the Company of Men (1997)* and *Misérables, Les (1995)*. This indicates the method's sensitivity to sparse or skewed user-rating data, which may lead to niche or less relevant outputs depending on the user's history.

Prompt- `recommend_movies_for_user(10, num=5)`

Movie Name	
Ranking	
1	In the Company of Men (1997)
2	Misérables, Les (1995)
3	Thin Blue Line, The (1988)
4	Braindead (1992)
5	Boys, Les (1997)

- **Item-Based Collaborative Filtering** focused on the similarity between movies, based on user rating patterns. Given a user's preference for *Interstellar*, the system returned related science fiction films such as *Gravity*, *The Martian*, and *2001: A Space Odyssey*. When applied to *Jurassic Park (1993)*, the top five recommendations included high-action adventure films like *Top Gun (1986)* and *Indiana Jones and the Last Crusade (1989)*, showing strong genre and theme alignment. This approach demonstrated robust genre consistency and higher accuracy for content-based recommendations.

Prompt-

```
recommend_movies("Jurassic Park (1993)", walk_length=10,  
num=5)
```

movie_name	
ranking	
1	Top Gun (1986)
2	Empire Strikes Back, The (1980)
3	Raiders of the Lost Ark (1981)
4	Indiana Jones and the Last Crusade (1989)
5	Speed (1994)

- **Random-Walk-Based Collaborative Filtering (Pixie Algorithm)** used graph-based exploration starting from either a user or a movie node. For a user-based query (`weighted_pixie_recommend(1, walk_length=15, num=5)`), the system recommended a diverse set of well-rated classics and popular titles, including *Toy Story (1995)* and *When Harry Met Sally... (1989)*. For movie-based walks starting from *Jurassic Park (1993)*, the algorithm surfaced a mix of genre-relevant films like *Twister (1996)* and classic cinema such as *High Noon (1952)*. This method effectively captured deeper relationships in the user-movie graph, balancing popularity and content diversity.

Overall, **Item-Based Collaborative Filtering** produced the most thematically aligned results for specific movie queries, while **Pixie's Random Walks** added diversity and graph-level exploration. **User-Based Filtering** was most sensitive to the quality and volume of individual user data. These findings highlight the trade-offs between personalization depth and recommendation relevance across different methods.

Prompt- `weighted_pixie_recommend(1, walk_length=15, num=5)`

```
weighted_pixie_recommend("Jurassic Park (1993)",  
walk_length=10, num=5)
```

```

User-based recommendations:
      Movie Name
1  When Harry Met Sally... (1989)
2           Toy Story (1995)
3           Jurassic Park (1993)
4           Amadeus (1984)
5  Remains of the Day, The (1993)

Movie-based recommendations:
      Movie Name
1  Jurassic Park (1993)
2    Piano, The (1993)
3    Twister (1996)
4    Brazil (1985)
5    High Noon (1952)

```

5.2 Comparative Evaluation

Accuracy

- **User-Based Collaborative Filtering:** Achieves high predictive accuracy when user neighborhoods are dense and preference overlap is strong. However, its performance degrades in sparse user-item matrices due to limited neighbor information.
- **Item-Based Collaborative Filtering:** Provides stable and consistent recommendations, especially for frequently rated items. Its reliance on item-item co-occurrence patterns can overlook niche or emerging interests.
- **Pixie (Random Walk Algorithm):** Offers a balanced trade-off between accuracy and diversity by leveraging multi-hop relationships in the user-item graph. Its effectiveness depends on the connectivity and density of the graph, potentially leading to degraded accuracy in sparsely linked subgraphs.

Usefulness

- **User-Based & Item-Based Methods:** Deliver predictable and interpretable results aligned with traditional collaborative filtering paradigms. Suitable for users with well-established preference histories.
- **Pixie Algorithm:** Generates serendipitous and exploratory recommendations by capturing latent connections in the graph. Particularly useful in discovery-oriented scenarios or when user history is limited.

5.3 Limitations

Despite producing relevant recommendations, the approaches explored in this project come with several limitations. The **User-Based Collaborative Filtering** method struggles with **sparse user-item matrices**, especially when users have rated only a few items, leading to unreliable similarity computations. Similarly, **Item-Based Filtering** can become biased toward popular items, often overlooking niche content. The **Pixie algorithm**, while effective in exploring deeper relationships, heavily depends on the structure and density of the interaction graph—users with limited interactions may receive less personalized suggestions. None of the models account for **temporal factors**, meaning they treat all ratings as equally relevant regardless of when they were made. Additionally, these methods lack **context-awareness** (e.g., mood, time of day, or device used), which could be vital for improving personalization.

VI. Conclusion

This project explored three collaborative filtering techniques—User-Based, Item-Based, and the Random-Walk-Based Pixie algorithm—to generate personalized movie recommendations. Each method demonstrated distinct strengths: User-Based filtering leveraged patterns from like-minded users, Item-Based filtering emphasized consistency through item similarities, and the Pixie algorithm uncovered nuanced connections through graph-based traversal. While the models performed reasonably well, several innovative enhancements could further improve their effectiveness. For instance, integrating **temporal dynamics**—such as giving more weight to recent ratings—could better reflect evolving user preferences. Additionally, introducing a **serendipity score** into the ranking process could help balance relevance with discovery, encouraging users to explore content they wouldn't typically choose. Another promising direction involves **session-based filtering**, which adapts recommendations based on a user's short-term behavior within a single interaction. Finally, combining recommendation outputs with **natural language explanations** (e.g., "People who loved *Inception* also enjoyed this for its mind-bending plot") could increase trust and transparency. These enhancements, when applied in real-world systems like streaming platforms, e-commerce, or educational tools, have the potential to drive deeper engagement and a more satisfying user experience.